

Boole-ANN: Joint Physical Design for Structured Filtered Vector Search

Qionghao Huang
qhhuang@zjnu.edu.cn
Zhejiang Normal University
Jinhua, China

Feiyang Shu
feiyangshu@zjnu.edu.cn
Zhejiang Normal University
Jinhua, China

Changqin Huang
cqhuang@zju.edu.cn
Zhejiang University
Hangzhou, China

ABSTRACT

Filtered approximate nearest neighbor search retrieves vectors that are similar to a query while enforcing structured predicates over their attributes. It is increasingly used in retrieval, recommendation, and semantic search, where a relevant result is useful only if it also satisfies application constraints. Existing systems, however, optimize either a collection of filtered indexes, the execution of complex predicates, or a generic fallback, leaving reusable predicate fragments, their physical layouts, and their query plans to be designed separately.

We introduce *Boole-ANN*, a physical designer that makes these decisions jointly. *Boole-ANN* learns a shared representation of exact predicate supports from a representative workload and uses it to capture equality atoms, recurring conjunctions, and ordered blocks. For every reusable fragment, it considers alternative physical layouts and admits only the combinations that are useful under the available resource budget. Ordered fragments can be composed to answer previously unseen ranges and Boolean expressions without materializing every query endpoint. At runtime, *Boole-ANN* selects a recall eligible plan, validates the predicate, and invokes a complete exact fallback whenever no specialized plan is appropriate.

We establish the correctness of the support algebra and exact execution tier. On real and controlled workloads, *Boole-ANN* outperforms state-of-the-art methods at protocol-matched, recall-qualified operating points for evaluated equality, conjunction, and range workloads while preserving exact predicate validity under explicit resource and tail-latency goals.

KEYWORDS

filtered approximate nearest neighbor search, vector databases, physical design, graph indexes, Boolean predicates, query optimization

PVLDB Reference Format:

Qionghao Huang, Feiyang Shu, and Changqin Huang. Boole-ANN. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/qhhuang-ai/booleann>.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Structured filtered ANN. Approximate nearest-neighbor (ANN) search returns the database vectors closest to a query vector. Structured filtered ANN additionally restricts the answer to records satisfying a Boolean or numeric predicate. A correct result must therefore satisfy two different contracts: every returned record must obey the predicate, while the ranked top- k should approximate the nearest valid records. Modern vector databases use this operation to combine semantic similarity with tenant, category, time, price, or other metadata constraints. Throughout the paper, the supported grammar comprises equality predicates, conjunctions of two equality atoms, one-dimensional closed ranges and their complements, and unions of two ranges on the same attribute.

Predicates as physical design. Database workloads repeatedly ask related predicates over one vector collection: a frequent equality, a hot conjunction, and many intervals with different endpoints. Their valid-record sets (their *supports*) often share atoms and subexpressions, but have very different cardinalities. That structure creates a physical-design opportunity. A support may be scanned as packed vectors, intersected as postings or bitmaps, or searched through a local ANN graph; the best choice changes with its size, reuse, and recall target.

Physical-design goals. An effective workload-aware designer should simultaneously provide (G1) reusable complete supports rather than cached query answers, (G2) composition for unmaterIALIZED conjunctions and unseen range endpoints, (G3) alternative physical layouts for one logical support, (G4) recall-eligible plan choice with predicate validation and a complete exact fallback, (G5) explicit persistent, build, and serving-memory budgets together with a tail-latency objective, and (G6) execution competitive with the strongest applicable filtered-ANN system as predicate form, cardinality, and recall change.

Design contentions. Existing systems meet important subsets of these goals. A universal label-aware graph serves unpredictable predicates but pays for filtering inside navigation; a predicate-specific graph removes that filtering but cannot be built for every conjunction or range endpoint. Workload-selected index collections choose among reusable subindexes, while complex-filter planners compose attribute indexes and range systems exploit order structure [6, 10, 20, 21, 25]; none of these choices makes one physical representation dominate at every support cardinality. Materializing every useful subexpression is exponential and can exhaust storage, whereas choosing from test identities would reduce physical design to a cache. Finally, approximate specialization may be fast without certifying complete predicate support, while an exact scan is

safe but often too slow. We therefore ask: *can one budgeted physical design share exact predicate fragments, choose their layouts and recall-eligible plans, and compose previously unmaterialized structured predicates without sacrificing competitive execution?* This paper presents an affirmative answer.

Core contribution: Boole-ANN. Boole-ANN is a physical designer that treats reusable predicate supports, their layouts, and executable plans as one design object. Equality atoms and endpoint-independent ordered blocks form the leaves of a shared exact-support DAG; intersection, union, and difference compose them to answer unseen two-atom conjunctions and ranges with unseen endpoints without caching query answers. The offline designer chooses among postings, bitmaps, packed vectors, and local graphs under storage, build, and serving-memory budgets and a tail-latency SLO. At runtime, the executor covers a supported predicate with frozen supports, selects an exact or recall-eligible plan, checks the original predicate, and reranks by exact distance; when no specialized plan is selected, it invokes a complete exact fallback. Boole-ANN thus couples reusable Boolean structure, physical layout, and predicate-valid execution for equality, two-atom conjunctions, one-dimensional ranges and complements, and same-attribute two-clause range DNF.

Additional contributions.

- We formalize exact support composition and prove predicate validity for complete covers and residual checks; a disjoint fixed-block family composes any one-dimensional interval with exactly N stored memberships in total.
- We implement a workload- and SLO-aware advisor that jointly selects fragments, layouts, and plans under explicit budgets, and a manifest-driven executor with startup validation and exact fallbacks.
- We evaluate Boole-ANN against released baselines on 10^5 – 10^8 vectors. Reported protocol-matched, recall-qualified comparisons cover equality, two-atom conjunction, one-dimensional range, and same-attribute two-clause range DNF; controlled interventions isolate materialization, layout, and routing effects.

2 RELATED WORK

Workload-driven physical design. Database physical-design tools use a representative workload, optimizer cost, and a storage constraint to choose indexes [5]; joint view/index selection expands that candidate space to reusable materialized results [1]. Boole-ANN imports this separation between logical workload and budgeted physical state into structured vector search: support fragments are the candidates, while posting joins, packed scans, and graph traversals are alternative layouts/operators. Section 3 formalizes this connection and the resulting selection objective.

Global label-aware graphs. Filtered-DiskANN [7], ACORN [14], UNG [4], and JAG [23] modify graph construction or traversal so one structure can serve many predicates. This is attractive when the query vocabulary is unpredictable, but it keeps predicate checking inside approximate navigation. Boole-ANN is complementary: for reusable supports it removes filtering from the inner graph entirely; requests without a specialized support use the generic complete-support fallback. Recent database-oriented

designs broaden this shared-structure lane: NaviX integrates predicate-agnostic search into a graph DBMS [16], RWalks diffuses attributes with graph walks [2], FAVOR uses selectivity-aware exclusion distances [18], MCI compresses a dense filter-agnostic neighborhood through maximal cliques [26], and Tag-Filtered ANN studies per-tag, unified, and hybrid graphs [13]. These strengthen the generic side of the design space; our evaluated point is the complementary specialization point, spending a declared budget on reusable exact supports while retaining the generic fallback.

Index collections and complex-filter planning. SIEVE [10] is the closest offline-design prior: it builds an HNSW subindex collection from a historical predicate workload, tunes graph density under a memory/recall constraint, and at query time picks one subsuming subindex or brute force; an appendix set-cover extension that unions exact-covering subindexes for disjunctive filters is important overlap with multi-asset composition, but those assets keep one graph layout and never materialize reusable support intersections or pick among posting, bitmap, packed, and graph layouts for the same support. VecFlow [22] targets GPUs with a label-specificity-dependent dual layout for one filter at a time. Its central design axis is hardware-dependent layout selection, whereas Boole-ANN studies reusable cross-predicate supports under a CPU serving budget. PathFinder [21] plans online over DNF-converted filters by selecting user-chosen subsets of attribute-specific proximity graphs rather than alternative layouts for one exact support under a workload/budget advisor. Compass [25] executes arbitrary mixed predicates by feeding one shared candidate queue from a vector graph and clustered relational indexes, using a broad predicate grammar and DBMS-integrated execution. Parlay-IVF [17] combines cluster and posting state for filtered search. Boole-ANN’s design point couples offline cross-predicate support/layout selection with online checked composition. The pair-removal, support-coverage, and budget controls in Sections 6.3 and 6.4 quantify the contribution of that coupling.

Ordered predicates. The B-Window Search Tree (β -WST) [6], SeRF [29], iRangeGraph [24], WoW [20], and RNSG [28] specialize navigation for numeric intervals. β -WST is particularly close to our ordered leaves: it recursively partitions the label order and composes several ANN indexes whose windows cover a query. Boole-ANN shares fixed blocks with Boolean supports, assigns them alternative exact/graph layouts, and selects checked plans from the combined portfolio. One disjoint fixed-block family has exactly N memberships and composes unseen endpoints, complements, and two-clause range DNF, at the cost of probing multiple blocks. Mesh [19] is already workload-aware and memory-budgeted for spatial range-constrained ANN, but its candidate and execution space is specialized to geographic meshes. DSG [15] and DIGRA [9] add dynamic segmented and graph structures for scalar range filtering. KHI [27] combines an attribute-space partition tree with node-local HNSW graphs for multi-attribute boxes; GPU-oriented Garfield builds cell-local graphs and a streaming execution pipeline [11]; and MSTG [12] handles containment and overlap when each object itself carries an interval. These systems extend the range-design space toward multidimensional, GPU-resident, and interval-valued

data. Our experiments use the available official scalar-range implementations for direct comparison and use same-kernel leaf controls to isolate mixed-DNF composition.

System integration. Recent system-level evidence on FAISS, Milvus, and pgvector shows that hybrid exact/approximate execution and optimizer choices can dominate the underlying ANN index [3]. PipeANN-Filter explores a verified superset through an SSD-oriented speculative-filtering design [8]. Boole-ANN operates at the physical-design and execution layer for a page-resident vector collection; relational transactions, updates, and SSD-oriented scheduling remain responsibilities of the surrounding engine.

Boole-ANN combines materialization, workload-driven selection, DNF planning, and cardinality routing around a new physical-design unit: a compositional exact support with alternative layouts and authorized plans, whose support algebra, budgeted selection, validity semantics, and disjoint design/evaluation protocol make the joint design executable.

3 PHYSICAL-DESIGN MODEL

Let $D = \{(x_i, a_i)\}_{i=1}^N$ contain vectors $x_i \in \mathbb{R}^d$ and structured attributes a_i , and let $\delta(q, x_i)$ be the dataset’s distance. Predicate φ defines support $X_\varphi = \{i : \varphi(a_i) = 1\}$. With $k_\varphi = \min(k, |X_\varphi|)$, the reference answer $T_k(q, \varphi)$ is the first k_φ IDs in the stable $(\delta(q, x_i), i)$ order on X_φ ; strict recall of output $\hat{T}$ is $|\hat{T} \cap T_k|/k_\varphi$ when $k_\varphi > 0$ (and one for a correctly empty answer). We report the distance-threshold metric separately when integer-valued distances tie.

The design vocabulary follows one chain: predicate φ defines an ID set X_φ ; a *fragment* is a reusable logical node for such a set; an *asset* realizes one fragment in a physical layout; a *plan* combines assets with operators; and a *manifest* is the budgeted set of assets and plans authorized for execution.

A logical *fragment* f denotes a reusable exact support P_f . Boole-ANN represents fragments as a hash-consed DAG:

- atom f_a with $P_{f_a} = X_a$;
- fixed block $f_{g,j}$ with $P_{f_{g,j}} = \{i : A(a_i) \in I_{g,j}\}$ for a candidate disjoint partition family g ;
- internal node $f = u \cap v$, $u \cup v$, or $u \setminus v$, whose support is the corresponding exact set operation on P_u and P_v .

Syntactically equal subexpressions share one node. The miner extracts subexpressions from calibration predicates, adds base-derived atom and block leaves, and rejects candidates below a minimum reuse or above a declared expression-size cap. The candidate set is fixed before evaluation; an uncovered expression is compiled from available leaves. A recurring pair is simply the node $f_a \cap f_b$, not a special-case index type.

From fragments to executable plans. A fragment is logical state, not yet an index. A physical asset $j = (f, \ell)$ couples it to a layout, and an executable candidate (j, o_θ) adds a compatible operator and frozen parameters. A plan may consume several assets, but every plan ends with deduplication, an exact predicate check, and a global exact-distance top- k merge. Table 2 makes the separation concrete. For example, $a \wedge b$ can use a materialized pair graph, filter a primary posting through a secondary bitmap, or merge two sorted postings; those are three physical plans for one logical predicate, not three unrelated systems. Joint selection chooses fragment

admission, physical layout, and executable plan in one budgeted optimization, allowing savings from one decision to be reinvested in the others.

A physical asset $j = (f, \ell)$ uses a layout $\ell \in \{\text{posting, bitmap, packed, graph}\}$. A stored-ID asset is complete only when its checked local/global map is a bijection onto the fragment’s declared exact support; this *completeness invariant* is independent of ANN recall. Candidate j has a measured resource vector $c_j = (\text{persistent bytes, build seconds, serving bytes})$.

For each workload class q , the enumerator produces plans p . A plan names its required optional assets $R(p)$ and its operator sequence; every class retains an always-available exact fallback plan. The calibration harness measures latency and freezes a conservative planning cost L_{qp} . Approximate candidates are assigned before evaluation to an admission group $a(q, p)$ (predicate family, operator, and the declared support stratum when available); each exact plan is an always-eligible singleton group. An approximate group is eligible only when the lower bound for the complete plan—including union, deduplication, predicate checking, and global reranking—meets target ρ_a on calibration. This is a distributional guard over formulas in the group. Let the fixed indicator $e_a \in \{0, 1\}$ equal one for exact singleton groups and for approximate groups that pass this guard. Binary x_j selects a physical candidate and z_{qp} selects one eligible plan. Let $s_{qp} = 1$ when the plan’s conservative latency estimate meets tail SLO τ , and let $\eta \in [0, 1]$ be the required workload-mass coverage. With nonnegative workload frequencies w_q and resource budgets B , let $\mathcal{J}_0$ contain assets not coupled to a block family. The designer solves

$$\begin{aligned}
& \min_{x,z} \quad \sum_q w_q \sum_{p \in \mathcal{P}_q} L_{qp} z_{qp} \\
& \text{s.t.} \quad \sum_j c_{jr} x_j \leq B_r \quad \forall r, \quad \sum_{p \in \mathcal{P}_q} z_{qp} = 1 \quad \forall q, \\
& \quad z_{qp} \leq x_j \quad \forall q, p, j \in R(p), \\
& \quad x_j \leq \sum_{q,p: j \in R(p)} z_{qp} \quad \forall j \in \mathcal{J}_0, \\
& \quad z_{qp} \leq e_a(q, p) \quad \forall q, p, \\
& \quad \sum_q w_q \sum_p s_{qp} z_{qp} \geq \eta \sum_q w_q, \quad x_j, z_{qp} \in \{0, 1\}.
\end{aligned} \tag{1}$$

Here branch eligibility is insufficient: an approximate group passes only when its complete composed plan passes the frozen guard. Fallback plans have $R(p) = \emptyset$ and are always eligible, so every supported query remains executable and predicate-valid. Exact fallback also provides recall one; selecting an eligible approximate family enforces the empirical group-calibration target, while exact fallback retains recall one. When the tail-SLO constraint is infeasible at a tight budget, the advisor reports the attainable coverage together with the resulting objective. We call a design optimal only at zero solver gap. In the degree-one layout-alternative topology (each asset has one assignment edge and each query chooses one fragment’s layout), the substitution $x_j = z_{qp}$ reduces Eq. (1) to a multiple-choice knapsack over the active budget dimensions, plus the tail-coverage side constraint when it is active; with one byte budget and no tail constraint this is the classical single-budget multiple-choice knapsack, and Lagrangian reduced-cost pruning plus a zero-gap

Table 1: Relationship to prior systems by physical-design dimension.

Lane	Closest systems	Persistent grain	Native predicate scope	Relationship
partition/filter workload collection	Parlay-IVF SIEVE	clusters + postings selected HNSW subindexes	equality / two-atom AND flat AND/OR or paired ranges	categorical reference support-design reference
GPU filtered global graph	VecFlow ACORN, UNG, JAG, F-DiskANN, FAVOR, MCI	label-specificity dual layout one label-aware graph	equality / multi-label AND equality / arbitrary filters	GPU layout selection shared-index designs
complex planner	PathFinder, Compass	attribute graphs; graph + B-trees	Boolean DNF / arbitrary range	broader online grammar
scalar range	β -WST, SeRF, WoW, iRangeGraph, RNSG	window tree / range graph	one-dimensional interval	range references
range supplements	DSG, DIGRA	segmented / dynamic graph	scalar range / mixed selectivity	scalar layout designs
multi-attribute	Mesh, KHI, Garfield	mesh / partition forest / GPU cells	geographic / numeric boxes	multidimensional designs
distinct semantics DBMS/engine	MSTG NaviX, MoReVec study, PipeANN-Filter	interval-valued graph DBMS / engine / SSD graph	containment / overlap structured / arbitrary filters	adjacent range task engine integration
Boole-ANN	this work	exact support DAG + layouts	equality, two-atom AND, 1D range/DNF	joint support/layout/plan

Table 2: Representative fragment–layout–operator plans. Every plan checks the original predicate on global IDs before exact-distance merging.

Predicate Assets and operator	Recall semantics
a atom pack + scan	exact
a atom graph + beam	empirical
$a \wedge b$ intersection-node pack/graph	exact / empirical
$a \wedge b$ posting + bitmap semi-join	exact
$a \wedge b$ two postings + merge	exact
$[l, r]$ overlapping blocks + scan/beam	exact / empirical
DNF shared DAG cover + checked union	exact / plan-level empirical

residual core certifies the original optimum. Plans coupling multiple fragments remain in the general MILP. On query-free degree-one catalogues the reduction matches the original zero-gap objective and selected bytes through 10K candidates; Section 6.3 evaluates the degree-one 100K endpoint and a connected catalogue whose accelerated plans share two or three assets. Supports and exact byte costs come from base data; workload weights, latency and recall eligibility come only from the disjoint calibration split. Candidate block granularities compete with atom/pair layouts in the same budgets. A group variable selects all members of a candidate block partition together, and at most one partition family may be selected. Its disjoint supports satisfy $\sum_j |P_{f_j}| = N$ exactly. Everything selected by Eq. (1), including online operator parameters, is frozen before held-out execution.

For candidate block family g , let $G_g = \{(f_{g,j}, \ell_g) : j = 1, \dots, B_g\}$ and use a binary h_g to couple the whole partition:

$$\begin{aligned}
 x_j &= h_g & \forall j \in G_g, \forall g, \\
 h_g &\leq \sum_{q,p: R(p) \cap G_g \neq \emptyset} z_{qp} & \forall g, \\
 \sum_g h_g &\leq 1, & h_g \in \{0, 1\} \quad \forall g.
 \end{aligned} \tag{2}$$

Thus selecting $h_g = 1$ charges every member of G_g in every applicable budget even if calibration touches only part of the domain. Resource coefficients are incremental over mandatory base postings

and counted once when an asset supports several plans. The design manifest records selected assets, enabled plan templates, operator parameters, cost coefficients, and fallback rules. Online execution may inspect predicate structure and immutable base support counts, but may not create a new pair, bitmap, block boundary, or beam setting.

PROPOSITION 1 (VALIDITY AND EXACT TIER). *Suppose every selected unit satisfies the completeness invariant and the executor evaluates φ exactly on every candidate before output. The units searched may be subsets or supersets of X_φ ; the executor returns only predicate-valid IDs. If exact enumeration of the selected units produces a candidate set C with $X_\varphi \subseteq C$, exact distance ranking of $\{i \in C : \varphi(a_i) = 1\}$ returns the exact filtered top- k . Replacing enumeration by graph traversal preserves validity but makes ranked recall empirical.*

PROOF. Completeness makes every local-to-global ID a member of its declared support; the final exact predicate test removes every ID outside X_φ , including IDs introduced by a superset atom or a boundary block. Thus output validity does not require a unit itself to be contained in X_φ . If $X_\varphi \subseteq C$, filtering C by φ yields exactly X_φ ; exact global top- k ranking is therefore sufficient. Graph traversal may omit valid members of C , but the final predicate test still precludes invalid output. $\square$

PROPOSITION 2 (EXACT SCREENING BY A CERTIFIED LOWER BOUND). *Let C be the complete predicate support after any leave-one-out exclusion, and let $\underline{\delta}(q, x) \leq \delta(q, x)$ for every $x \in C$ with respect to the same exact scorer. Let $S \subseteq C$ be a fully scored seed with $|S| \geq k$, and let τ be the k th distance in the stable exact order on S . Discarding only rows with $\underline{\delta}(q, x) > \tau$ preserves the stable exact top- k of C . The result remains exact when a failed bound, nonfinite value, insufficient shortlist, or cutoff tie invokes full exact enumeration of C .*

PROOF. The global k th distance in C is at most τ . A discarded row has exact distance strictly greater than τ and therefore cannot enter the top- k ; the strict test retains every cutoff tie. Full enumeration is exact by Proposition 1. $\square$

The online optimizer evaluates only plans enabled by the selected design. It estimates T_{join} from posting/bitmap reads, $T_{\text{scan}} \propto |P|d$, and T_{graph} from support, beam, and requested k , then chooses the

least-cost recall-eligible plan. Estimates use observable base cardinalities, never query outcomes; all coefficients and beams are calibrated once and frozen. The evaluation reports calibration/test separation, resource coverage, and forced materialization controls.

Calibration and conservative eligibility. We stratify calibration observations by predicate form, support decile, and operator. For joins, features are the two posting lengths, intersection estimate, bitmap presence, and packed-vector availability; scan features are candidate count and dimension; graph features are support, requested k , and beam. Robust log-linear fits predict the median, while an empirical upper-residual correction supplies the planning cost used by the designer. For an approximate plan, recall eligibility is a separate decision: its complete end-to-end plan’s one-sided lower confidence bound must exceed the requested target. Latency and recall are independent gates, so a low estimated cost does not establish eligibility. Exact plans are eligible by construction.

For SIFT100M, eligibility uses 200 unique calibration queries from eight predeclared pair classes (25 each). For each of the materialized pair-graph and pair-disabled fallback families, a one-sided 95% Student- t lower bound across the eight class-mean strict recalls must reach .99. These two pooled family guards are the candidate-testing boundary: fragments inherit the family decision, with no fragment-wise fit or post-hoc family expansion. Admission, beams, and plan parameters are fixed before the 58 held-out pair classes are evaluated. This screen governs group-level empirical ranked recall; support completeness, final predicate checking, and exact-fallback ranking retain the deterministic contract of Proposition 1. We report held-out per-class recall separately as a transfer diagnostic.

For the coverage-first study in Section 6.3, we first maximize the left-hand side of the coverage constraint under the same budgets and set η to that attainable optimum. Among coverage-optimal designs, calibration CVaR₉₅ and then the mean objective in Eq. (1) break ties. In deployments where η is an operator-specified SLO, only the latter stages are needed. The mean stage can equivalently maximize a conservative saving Δ_{qp} relative to the mandatory fallback, so a candidate with an uncertain or non-positive lower-bound saving has no incentive to consume space. This coverage–tail–mean ordering is important: optimizing additive mean saving can leave slightly more than 5% of workload mass on a slow exact fallback and therefore miss the p95 SLO even when two additional fragments would cross the coverage threshold. Support and byte features come from the base, whereas frequencies and fitted performance come only from calibration. Development chooses the global budget and recall targets; the held-out workload is used only once for the reported evaluation.

4 SYSTEM

Figure 1 makes the new design object concrete. The offline designer may inspect base attributes and a declared calibration workload, but not development/final query vectors, ground truth, or final outcomes. It freezes a shared support DAG, selected layouts, and operator parameters. At serving time an unseen formula is covered by those frozen nodes; the executor may warm pages but cannot add an intersection or move an ordered breakpoint, and this transient working set is included in peak RSS. We call the evaluated

equality/conjunction implementation the *Boolean Conjunction Index* (BCI); the ordered-range implementation (Predicate-Cube, X3 in the evaluation) uses the same materialization-unit and checked-merge interface. They are executor backends of one design contract: both consume manifest rows that name complete supports, charged layouts, authorized plan templates, and exact fallbacks; only the predicate-specific compiled operators differ.

4.1 Offline units and admission

The designer hash-conses atom and fixed-block leaves plus recurring calibration subexpressions, computes their exact supports by posting algebra, then instantiates each eligible support–layout alternative. A candidate stores a complete local-to-global ID map; a cardinality assertion rejects partial or mixed builds. Short supports may receive contiguous vectors for exact SIMD scanning, high-support membership tests may receive bitmaps, and long reusable supports may receive an ordinary Vamana graph. Every internal node is built by an exact set operation: YFCC’s selected intersection plans use packed exact layouts, whereas the SIFT100M causal experiment makes graph layout an alternative. Block widths and graph/packed layouts are candidates in Eq. (1), rather than post-hoc implementation choices. Common-subexpression reference counts ensure that a materialized node is charged once even when several formulas consume it.

SIFT100M isolates one DAG layer: its candidate universe is the 66 intersections induced by 12 base labels. We build every candidate once to permit counterfactual replay, but a deployed design loads and is charged only for selected graph and ID-map files. A representative calibration slice over the same predicate vocabulary estimates workload mass and plan latency; query vectors are disjoint from the held-out slice. Evaluation spans the complete non-benchmark pair universe across three orders of support magnitude; YFCC separately tests online composition of unmaterialized conjunction identities. A coverage/CVaR-aware formulation first satisfies the declared tail-latency coverage SLO and then maximizes conservative mean saving. This experiment tests non-benchmark queries over the predeclared 12-label pair universe. The evaluation reports no-pair, all-pair, benefit/byte, and smallest-cost controls. The YFCC workload’s older frequency-capped pair store admits all 3,335 observed calibration pairs and serves as an ablation. For ranges, alternative fixed-block families compete in calibration; the selected 200 width-25 blocks build one $R=32$ graph per block, so every point is represented once.

The universal conjunction fallback is a base-derived candidate family. Let $T = 2^{16}$. It stores an N -bit bitmap exactly for labels with base support greater than T (only 53 labels and 66,250,000 bytes on YFCC10M). A pair is oriented by (support, label ID). If the secondary support exceeds T , its bitmap filters the primary posting; otherwise both postings are bounded and a streaming merge emits their exact intersection. Primary-local offsets use 16 bits only when support is at most T ; the high–high case uses 32-bit global IDs and cannot truncate offset 65,536. This construction covers every label pair and uses no query vocabulary. A separately budgeted 824-label development vocabulary remains an optional accelerator, but it is not needed for correctness or coverage. The complete-q30 exact endpoint enables 128 calibration-selected hot atoms in addition to

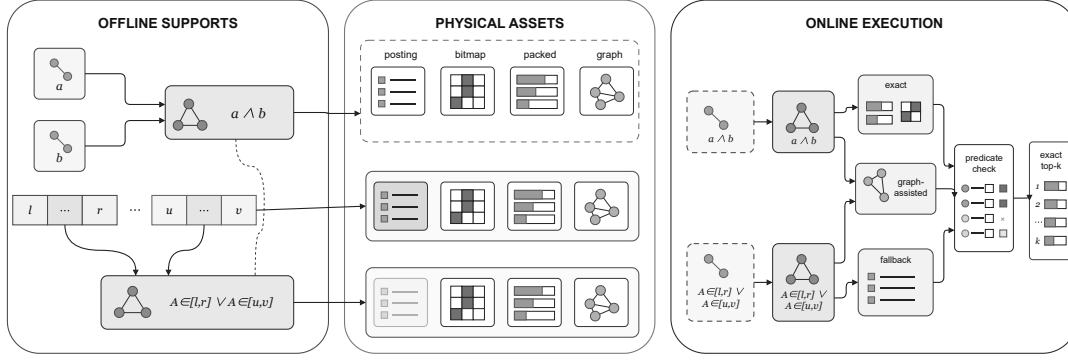


Figure 1: The Boole-ANN design object. Atoms and endpoint-independent blocks form one exact support DAG; internal supports can be shared by formulas and assigned alternative layouts. The categorical conjunction $a \wedge b$ and the separate two-clause range DNF $A \in [l, r] \vee A \in [u, v]$ are examples from the implemented grammar that reuse the displayed nodes. Calibration selects fragment–layout–plan triples under storage, build, serving-memory, and tail goals. The frozen manifest enables exact or recall-eligible plans, while every route ends in the original predicate check and exact-distance merge.

the required high-support family; overlap leaves 148 bitmaps and approximately 185 MB.

4.2 Online plan selection

For an equality atom, the executor uses its unit graph when admitted and an exact posting scan otherwise; the LAION scan may add a base-norm screen whose outward-rounded lower bound follows Proposition 2, with full exact enumeration on any certificate failure. For $a \wedge b$, it first checks the frozen pair dictionary. A hit executes that pair’s declared exact or graph layout. On a miss it invokes the universal support-complement intersection above, then scans contiguous vectors when the primary atom is packed and otherwise prefetches only the exact surviving global vectors. When a selected unit graph is recall-eligible, the optimizer may instead use its frozen beam. If a required unit is absent, execution falls back to exact intersection; it never treats an absent unit as complete. Every returned ID is predicate-checked and all surviving candidates are reranked by the original exact distance.

Table 3: Calibrated cost surface for support P ; m is the emitted exact intersection and B is a graph beam. “Exact” means ranked top- k .

Plan	Charged state	Dominant query work	Contract
posting merge	sorted IDs	$ P_a + P_b + md$	exact
bitmap join	$N/8$ -byte bitmap	$ P_{pri} + md$	exact
packed scan	IDs + $ P d$ scalars	$ P d$ contiguous	exact
norm- screened scan	one norm/row	base bounded candidates + exact distance	exact
unit graph	ID map + $R P $ edges	Bd + checked merge	empirical
block cover	one partition, RN edge slots	sum of covered units	exact if scanned

The table predicts crossovers rather than a uniformly best layout: merge pays when both postings are short, a bitmap removes the secondary-posting scan, packed vectors win while complete support enumeration remains cache-friendly, and a recall-eligible graph caps distance work on long supports. Calibration fits the

omitted machine constants. Section 6.4 isolates the causal value of admitting one exact pair fragment and holds the logical range supports fixed and changes only their retained versus hybrid layout. The two interventions therefore measure support admission and layout selection, respectively.

The final YFCC design illustrates why the plan, rather than any one ANN structure, is the system contribution. Single-tag queries divide between selected Vamana units and exact scans; every two-atom query uses an exact posting–bitmap or posting–posting join followed by packed/gather scan. Conjunctions use only base-derived posting and bitmap state. Thus all conjunctions are exact and every ranked-recall loss in the mixed workload comes from selected single-tag graph plans.

4.3 Ordered composition

A range decomposes into complete interior blocks and at most two partial boundaries. The executor searches each overlapping unit, rejects out-of-range boundary candidates, and merges the local pools by exact distance. A union of two decompositions over the same numeric attribute executes the implemented two-clause range DNF; a one-dimensional negated interval is the union of its two complements. The index is independent of endpoints, and its total graph membership remains exactly N . Searching only a beam from each block makes recall empirical; enumerating the same block cover gives the exact tier. Approximate DNF eligibility is measured after the entire union, deduplication, predicate check, and global rerank; it is not inherited from per-clause recall.

4.4 Cost surface and fallback behavior

The materialization costs are explicit. An atom or pair graph on support P stores $R|P|$ directed edge slots plus its ID map; a packed exact pair stores $|P|$ IDs and $|P|d$ scalar coordinates; a dense membership bitmap costs $N/8$ bytes independent of tag frequency; and the complete fixed-block family stores RN edge slots because its supports partition the base. These quantities, not logical predicate count, are the capacities in Eq. (1).

For an uncached conjunction with sorted postings P_a and P_b , the exact fallback is a merge semi-join in $O(|P_a| + |P_b|)$ integer reads and computes distances only for intersection members. If the secondary bitmap is present, it instead scans the chosen primary in $O(|P_{\text{pri}}|)$ membership probes. The exact top- k heap adds $O(|P_a \cap P_b| \log(k+1))$ work. Approximate unit search replaces enumeration with a bounded beam but retains the complete ID map and final predicate check. A missing or corrupt unit activates the slower declared complete fallback.

4.5 Manifest validation and fallback semantics

The designer emits a manifest rather than executable source. Each row names a logical support, layout files, exact byte and edge charges, construction parameters, enabled operators, calibrated latency coefficients, recall eligibility, and a fallback plan. Shared assets appear once and are reference-counted by plans. Startup validates row counts, local/global ID ranges, graph degree, block disjointness, and required files before accepting queries. Optional-row validation failure disables its dependent plans; mandatory-posting validation failure aborts startup.

For each request the predicate compiler performs four steps: (1) match the request against the supported grammar defined in Section 1; (2) enumerate only manifest-authorized plan templates; (3) discard plans that lack frozen whole-plan calibration eligibility for the requested recall target or a required asset; and (4) choose the minimum predicted cost. Other formulas are rejected at the API boundary. Execution returns global IDs, removes duplicates across branches, evaluates the original predicate, computes exact distances, and applies a stable (distance, ID) top- k . If no specialized plan survives, the exact posting/scan fallback remains, converting a missing specialized path into additional latency while preserving complete-support recall.

Concurrency and accounting. One serving instance owns the index and eight worker-local scratch regions; queries are distributed dynamically, while each individual query executes on one worker. Consequently batch QPS measures eight-query offered concurrency, not an eight-thread acceleration of one competitor query. Competitors that expose only a single-query entry point are wrapped by eight independent calls over the same read-only index; their core search code and parameters are unchanged. Planner time, result conversion, deduplication, predicate checks, and exact reranking are inside the query interval. Dataset loading, index reconstruction, and the post-run correctness audit are reported separately.

4.6 Implementation

One advisor emits the fragment–layout–plan manifest consumed by three execution backends: the categorical graph/posting engine (BCI), the contiguous Predicate-Cube configuration (X3), and the ordered-block engine for ranges and two-clause DNF. All three use the same plan authorization, global-ID predicate check, exact-distance merge, fallback, and resource-accounting contract.

The equality/conjunction engine uses $R=64$ Vamana unit graphs, sorted 32-bit postings, dense 64-bit-word bitmaps, and contiguous byte-vector packs. The selected YFCC exact plan orients each conjunction by (support, tag ID). Primary supports at most 2^{16} emit checked 16-bit local offsets; larger supports use 32-bit global IDs.

A boundary self-test rejects support 65,537 from the 16-bit path and verifies ID 65,536 through the global fallback. Eight workers own reusable, first-touched scratch buffers; allocation, build, and correctness audit are outside the measured batch, but every byte remains in peak RSS.

The range engine uses $R=32$ Vamana graphs over the selected disjoint blocks. All competitors retain their released core algorithms; adapters only translate the common vectors, predicates, thread count, and result IDs. Every system is run on identical query IDs and exact ground truth. Boole-ANN’s planning time is included in query latency; design/build time is reported separately.

For LAION, U24 stores each sorted global ID in three bytes. DELTA16 stores the first ID as a 32-bit integer followed by unsigned 16-bit gaps, and is eligible only when every gap is at most 65,535. Construction round-trips either encoding against the exact support and rejects an invalid or overflowing asset. A DIRECT asset stores one complete support, whereas FRAG composes endpoint-independent U24 blocks and checks the original predicate before scoring.

5 PHYSICAL-DESIGN GUARANTEES AND SCOPE

The deployed guarantees establish support correctness and exact-tier ranking; approximate graph plans use calibrated recall measured in §6. Build-time checks establish a bijection between every materialized unit’s local IDs and its declared posting. Consequently, atom and pair search cannot return a predicate-invalid ID; an exact scan over a complete atom or pair is exact filtered top- k . Posting merge and bitmap semi-join enumerate exactly the intersection they declare, so the support-complement path is exact regardless of the query-pair vocabulary.

PROPOSITION 3 (ENDPOINT-INDEPENDENT BLOCK COVER). *Let A be defined on every record, and let $I_1, \dots, I_B$ be a disjoint partition of an ordered domain containing every realized $A(a_i)$. Store $P_j = \{i : A(a_i) \in I_j\}$. Then $\sum_j |P_j| = N$. Any closed interval $[l, r]$ is the union of zero or more complete interior blocks and candidates from at most two boundary blocks; checking $l \leq A(a_i) \leq r$ on those candidates yields exactly its support. Complement relative to the base universe and finite union preserve exactness.*

PROOF. Every base value belongs to exactly one partition interval, establishing the membership sum. Every block strictly between the blocks containing l and r lies inside the query; only the two endpoint blocks can overlap without being contained. Exact endpoint checks remove their out-of-range IDs. Set-theoretic union, complement relative to $[N]$, deduplication, and the final predicate check therefore produce the exact requested support. $\square$

Exact enumeration of every block in the cover is therefore exact filtered top- k ; deployed graph traversal over the same cover preserves validity but has the empirical recall reported in §6.2. Notice that the proof depends on a complete partition family. This is why Eq. (2) selects all blocks of one candidate family together instead of optimizing individual intervals after observing endpoints.

The materialization budget is explicit: an atom or pair of support m costs Rm graph edges plus its ID map, while a fixed-block family costs RN edges. If a requested fragment is absent, the executor

does not infer completeness: it uses the exact posting/scan fallback. The deterministic guarantees therefore cover predicate validity and exact-tier ranking for the supported fragment vocabulary. Approximate graph plans use empirically calibrated recall, and workload optimization ranges over the candidate fragments, layouts, and operators in Eq. (1).

PROPOSITION 4 (FAIL-CLOSED ASSET REMOVAL). *For every predicate in the supported grammar, removing an optional asset from a valid manifest cannot make an output predicate-invalid or eliminate the exact tier: every dependent plan is disabled and the mandatory exact fallback remains. A replacement exact plan returns the same stable (distance, ID) top- k ; an approximate replacement retains its frozen group-level recall eligibility; and exact fallback has recall one.*

PROOF. Plan authorization requires every asset in $R(p)$. Removing one therefore prevents partial execution of p . For the supported grammar, the fallback has $R(p) = \emptyset$, enumerates a support superset containing X_p , and applies the original predicate and exact distance ranking, so Proposition 1 applies. Every remaining approximate plan has already passed the unchanged whole-plan calibration guard of Eq. (1). $\square$

6 EVALUATION

We organize the evaluation around three questions. **Q1** asks whether Boole-ANN outperforms the applicable state of the art across equality, conjunction, range, and two-clause DNF workloads. **Q2** asks whether the advisor chooses effective designs under a byte budget and a tail-latency SLO, including mixed serving. **Q3** asks how much fragment materialization, physical layout, and plan selection each contribute. Sections 6.2–6.4 answer these questions in that order after a common setup.

6.1 Setup

Platform and measurement. All experiments run on one AMD EPYC 7642 server with 96 cores, 251 GB RAM, and no swap. Unless noted otherwise, search uses eight workers over one resident index. QPS is completed queries divided by batch wall time; we also report request p50/p95/p99 latency and peak RSS where applicable. The SIFT and Parlay-IVF graphs use degree 64, while ordered range blocks use degree 32. Each comparison states its process, warmup, pairing, and inference unit.

Data and workloads. Table 4 covers natural metadata and controlled predicates. SIFT100M supplies an exhaustive 66-pair candidate universe and a 58-pair held-out evaluation; SIFT10M uses published predicates and held-out numeric endpoints. YFCC10M and LAION1M supply real tags and caption tokens.

Table 4: The workloads span real tags and captions plus controlled equality, range, and DNF attributes.

Dataset	N	dim	Predicate class
SIFT (BIGANN)	100K/10M/100M	128	controlled eq/range/DNF
YFCC10M	10M	192	real Flickr tags (100K q)
LAION1M	1M	512	real caption labels

Baselines and controls. Equality and conjunction experiments compare with Parlay-IVF and released SIEVE; range experiments compare with released WoW and SeRF. The global label-aware lane is represented by ACORN. The artifact also records the applicability, resource, and correctness gates for released UNG, iRangeGraph, Filtered-DiskANN, and JAG configurations. Competitors receive identical vectors, predicates, exact ground truth, offered concurrency, and warm-state policy. Adapters preserve the released search algorithms and only translate inputs, invoke independent queries, and convert returned IDs. The materialization control disables the pair fragment while holding atom state, degree, beam, and router fixed. For two-clause DNF, both systems issue two native interval searches, recheck the original clauses, and merge by exact distance; the WoW adapter invokes the same released search kernel for both clauses.

Splits, correctness, and resources. Base data determine candidate supports and byte costs; calibration selects designs, development fixes budgets and parameter grids, and held-out queries alter neither. All runs use exhaustive tie-aware ground truth and validate returned IDs. YFCC distances are recomputed with integer squared L2, an independent CPU implementation checks sampled supports and top- k answers, and a pre-timing control reproduces expected graph degree and baseline recall. We distinguish persistent index bytes from whole-process RSS/PSS, which also includes mandatory state, allocators, descriptors, and scratch; each experiment reports the matched endpoint in its native unit. Unless a table explicitly labels total memory, a byte cap applies to optional design state over mandatory base vectors and postings; total persistent state and process RSS/PSS are reported separately.

6.2 End-to-end performance across predicate families

Experiment 1: Natural tags against Parlay-IVF and ACORN.

We first vary the recall requirement on YFCC10M while holding queries, graph degree, worker count, exact ground truth, and warm-state policy fixed. YFCC follows BigANN’s public single- and two-tag protocol: qids [60K, 70K] calibrate the pair vocabulary, [0, 10K] choose the operating grid, and untouched [10K, 20K] queries form the primary test. We name 10K-query slices by starting index: q10 is [10K, 20K), q30 [30K, 40K), q80 [80K, 90K), and q90 [90K, 100K). Both systems serve page-resident batches, with construction, load, and first touch outside the measured batch.

The exact online-composition design contains 2,106 frozen primary packs occupying 4.642 GB and 148 base-derived bitmaps occupying approximately 185 MB; pair intersections are computed online. Figure 2 and Table 5 report the selected high-recall and exact endpoints against the strongest matched Parlay-IVF configuration.

Table 5: Selected YFCC10M endpoints against released Parlay-IVF.

endpoint	recall (Boole/PIVF)	QPS (Boole/PIVF)	ratio [95% CI]
held-out high recall	.99978/.99965	17,842/14,932	1.195 [1.191, 1.198]
complete q30 exact	1.000/1.000	28,881/13,700	2.108 [1.878, 2.366]

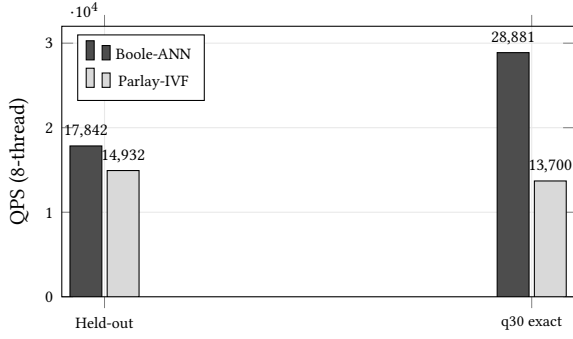


Figure 2: Boole-ANN outperforms Parlay-IVF at the selected YFCC10M held-out high-recall and complete-q30 exact endpoints.

The high-recall endpoint asks whether plan selection identifies an efficient recall-qualified operating point. Development q30 selects the smallest eligible policy—beam 192, raised to 224 only for supports of at least 800K points—and disables the costlier risk tier. On development data, this policy is $1.103\times$ faster than raising seven whole tag classes to beam 384 while retaining recall 0.99991 versus PIVF’s 0.99990; those raised classes add 32.9% distance computations and 55.1% visits per affected query for six additional hits among 100K result positions. After freezing the policy, we pair q80 vectors with a bijective, fixed-point-free permutation of q90 predicates and compute exhaustive ground truth. Across five opposite-order process pairs, Boole-ANN sustains 17.78–17.89k QPS versus 14.90–14.95k for Parlay-IVF R64, a $1.195\times$ geometric mean with 95% interval [1.191, 1.198]. Tie-aware recall is 0.99978 versus 0.99965 (strict 0.99977 versus 0.99964); all 18 passes per process return identical answers with zero predicate, duplicate-ID, or invalid-ID errors. The gain therefore comes from the least costly recall-eligible plan rather than uniformly greater graph effort.

A complete-workload endpoint evaluates all 3,825 public-q30 two-tag queries against pinned Parlay-IVF R64 P2. Three process-level ABBA campaigns use identical IDs, CPUs, and eight-worker budgets, materialize no query-pair row, and yield six ratios from 2.007 to 2.235. Their geometric mean is $2.108\times$ with campaign-level log- t 95% interval [1.878, 2.366]. Both systems reproduce identical top-10 outputs at recall one with no invalid or duplicate IDs; Parlay-IVF uses the same released binding and 12.04 GB index. The independently frozen q80/q90 comparison above supplies held-out inference.

At this matched approximate endpoint, Boole-ANN and Parlay-IVF occupy 12.038/12.040 GB persistently and average 24.312/24.381 million reported kB of peak serving RSS. For exact composition, the corresponding totals are 4.827/12.040 GB and 7.56/24.4 million reported kB.

We also compare exact execution with ACORN on a fresh 64-identity population containing 11 ACORN graph routes and 53 exact-prefilter routes (37 one-atom and 27 two-atom). Disjoint calibration freezes $ef=2048$, the first tested value reaching .995

recall. Five paired runs execute four fresh-process blocks in alternating order on eight CPUs; both timers exclude load and support preparation. Boole-ANN obtains 5.65–6.01k QPS versus 110.6–112.8 for ACORN, a $52.486\times$ geometric mean with 95% interval [50.582, 54.463]. Both systems attain recall one in every route/width stratum with zero errors; peak RSS is 23.284/12.336 million KiB and persistent index size is 12.038/11.826 GB (Boole-ANN/ACORN).

Forced plans on the same held-out q10 workload explain the portfolio result: exact scan gives 395 QPS at recall one, forced graph and cluster routes give 11,071/9,880 QPS at .91447/.92662 recall, and the frozen portfolio gives 12,607 QPS at .99947 recall. Sparse real-tag pair reuse thus favors base-derived atoms and online intersection; Section 6.4 tests the complementary high-reuse regime.

Experiment 2: SIEVE across four predicate families. We next hold a 400-query SIFT100K workload, a 24-MiB cap, and one selected Predicate-Cube package (X3) fixed while comparing with the closest workload-aware collection design. Queries are balanced across equality, conjunction, range, and DNF; query rows and conjunction/DNF identities are disjoint from design and earlier evaluation workloads. Five paired single-thread blocks alternate system order and rotate family order. Both systems consume identical uint8 vectors, SIEVE uses its released `PartitionedHNSW<int, uint8_t>` core, and design freezes $ef=100$.

X3 uses the implementation primitives of Section 4.6, orders one vector payload by (bin, A, B, id), initializes top-10 threshold τ with 256 full distances, and applies the safe bound $d(q, x) = d_{64}^{(0)}(q, x) + d_{64}^{(1)}(q, x) \geq d_{64}^{(0)}(q, x)$ to stable-compacted 128-row microbatches before evaluating the remaining 64 dimensions. Complete cube support is the exact fallback, and, including a 4-KiB transient bound, X3 charges 17,136,868 bytes (16.34 MiB) against the common cap. The cap bounds the optional design package; shared base state accounts for the remainder of each endpoint’s reported persistent total.

Table 6: Predicate-Cube is faster than SIEVE in every evaluated predicate family.

family	QPS X3/SIEVE	ratio [block range]	recall X3/SIEVE
equality	29,574/13,902	2.120 [2.115, 2.147]	1.000/1.000
conjunction	240,375/156,144	1.539 [1.513, 1.576]	1.000/1.000
range	5,301/3,597	1.474 [1.448, 1.502]	1.000/.998
DNF ₂	137,109/83,138	1.654 [1.609, 1.670]	1.000/1.000
mixed	17,100/10,857	1.574 [1.552, 1.599]	1.000/.9995

Table 6 gives the headline: the fixed package reaches a $1.574\times$ median mixed-workload ratio, and every one of the 20 family/block ratios exceeds one. X3 reproduces the oracle top-10, while SIEVE has .9995 pooled recall (.998 on ranges). The independent conjunction frontier is positive at all 11 $ef=100$ –200 points, with $1.557\times$ geometric-mean throughput and recall 1.000/.998909. At that frontier, Boole-ANN/SIEVE persistent state is 35.97/28.15 MB, preloaded process RSS is 43.85/101.28 MiB with zero swap, and preload cost is 5.26/12.49 CPU-s. Across equality, conjunction, range, and DNF, the same physical package therefore improves execution without weakening exact predicate validation.

Experiment 3: WoW and SeRF on range and DNF. At SIFT10M, two-clause DNF reuses 200 selected width-25 blocks and

exactly N graph memberships. We test 1,000 held-out intervals of widths 26, 251, and 501 against WoW at frozen beam/ef 160/1700, plus a fixed DNF workload. *Unseen* denotes interval endpoints: block coverage is exact, while graph ranking is recall-qualified end to end.

Table 7: Boole-ANN exceeds the 1.20 $\times$ margin on unseen SIFT10M ranges and DNF while both WoW comparisons meet the 0.99 recall target.

Predicate	recall (Boole/WoW)	QPS ratio	95% lower	p
Unseen range	.9965/.9944	1.710	1.682	< .001
DNF ₂	.9938/.9945	1.491	1.440	< .001

Across eight order-balanced range pairs, the geometric-mean gain is 1.710 $\times$; the family-wise one-sided 95% lower bound is 1.682 $\times$ against the prespecified 1.20 $\times$ margin ($p < .001$). Strict recall is 0.9965 versus WoW’s 0.9944 (tie-aware 0.9966/0.9945), and complete-process peak PSS is 6.01/14.90 GiB with zero swap.

DNF composes two range-clause candidate pools through predicate checking, deduplication, and global exact reranking. In five fresh processes on the fixed workload, all returned IDs satisfy the original predicate. The geometric-mean gain is 1.491 $\times$ and the one-sided 95% lower bound is 1.440 $\times$, above the 1.20 $\times$ margin ($p < .001$); recall is 0.9938 versus 0.9945, above the declared 0.99 target for both systems.

On the same intervals, SeRF at $ef = 8,192/16,384/32,768$ obtains 354.0/189.8/101.5 QPS at recall .9704/.9712/.9715 over five measured passes on eight isolated lanes. All IDs are valid and unique, and eight ranges contain fewer than ten results. Boole-ANN obtains .9965 recall and 2,070 QPS with 6.01/9.69 GiB peak PSS. The same 1.292 GB Boole-ANN block index serves range and DNF, versus 14.080 GB for WoW; matched builds take 553/7,491 s. These results extend the end-to-end comparison from categorical predicates to unseen ranges and composed DNF.

6.3 Budgeted design and mixed serving

Experiment 4: Fragment admission under a fixed budget.

We isolate the advisor from the complete serving path by fixing the search kernel and varying only which exact pair supports fit a 2.5 GiB budget. The workload contains 58 active SIFT100M pair classes and 100 logical queries per class in each of five processes. The first 20 queries per class estimate fragment value; the remaining 80 produce $58 \times 80 \times 5 = 23,200$ held-out replay observations. Selection uses calibration latency and exact fragment bytes, never test latency; a hash-based 20/80 split gives the same ordering.

The cheapest 56 active supports consume 2,620,392,060 bytes and fit, whereas the cheapest 57 consume 2,845,162,220 bytes and do not. Thus 56/58 = 96.55% is the maximum uniform coverage. The SLO-aware advisor first attains that coverage and then minimizes calibration CVaR₉₅ and mean latency among the ten feasible 56-support designs.

Table 8 shows that the selected design improves the reciprocal-mean-latency proxy by 1.470 $\times$ and reduces p95 by 12.17 $\times$ over the strongest calibration-only mean-saving policy; process-level

Table 8: At 2.5 GiB, coverage-first admission removes the p95 fallback tail left by benefit/byte selection.

Design	active	mean ms	p95 ms	proxy
no pair asset	0/58	83.738	98.758	11.94
old benefit/byte	54/58	9.136	85.695	109.46
max-coverage/cheapest	56/58	6.245	7.049	160.13
SLO-aware selected	56/58	6.217	7.041	160.85
all active (over budget)	58/58	3.326	6.564	300.66

p95 is 6.57–7.22 ms. Within the maximum-coverage set, SLO-aware tie-breaking further reduces both mean and p95 latency.

We separately test transfer over eight byte budgets. Policies are frozen from eight calibration pairs and replay all 100 queries in each of 58 other pair classes across five processes, giving $58 \times 100 \times 5 = 29,000$ observations per policy. For selected set S , held-out utility is the equally weighted positive mean-latency saving

$$U(S) = \frac{1}{58} \sum_{i \in S \cap H} \max(0, \bar{L}_i^{\text{off}} - \bar{L}_i^{\text{on}}),$$

where H is the 58-class held-out set and fragments selected outside H consume budget without adding held-out saving. U_{oracle} maximizes this post-test quantity under the same byte budget and is used only for evaluation.

Table 9: Across eight budgets, advisor utility remains within 5.59% of the held-out oracle.

budget	S	used MiB	$U(S)$	U_{oracle}	regret
0	0	0.0	0.00	0.00	0.00%
128 MiB	27	128.0	32.12	33.58	4.36%
256 MiB	38	236.5	47.18	47.18	0.00%
512 MiB	43	495.5	53.21	54.75	2.82%
1 GiB	48	1023.7	60.52	62.18	2.68%
2 GiB	58	2024.4	72.13	73.78	2.24%
3 GiB	63	3040.9	76.48	81.00	5.59%
all	66	3520.1	81.00	81.00	0.00%

At 1 and 2 GiB, held-out mean latency is 23.89 and 12.28 ms, versus 25.37 and 13.76 ms for the strongest frozen benefit-per-byte control. All eight HiGHS solutions reproduce the frozen selections exactly in 40.6 ms total (10.2 ms maximum per budget), with certified relative gap at most 1.6×10^{-16} . Real selected-manifest loads at 1–3 GiB agree with same-binary replay means within 3.83%, retain recall at least 0.99981, load in 10.19–12.01 s, and peak at 65.64–67.96 GiB RSS. The archived 70-shard R64 build takes 2,756.5 s and peaks at 49.33 GiB. Across all 30 real-loaded policy–budget points, median replay errors are 3.8% for mean and 6.1% for p95 latency (95th percentiles 9.3% and 19.7%); predicted-versus-loaded rank correlations are 0.993 and 0.696.

The advisor’s query-free scale controls preserve the same resource contract. At 100K degree-one candidates, the selected state is 5,787,289,400 B with 16 B slack; presolve reduces 600K variables and 700001 constraints to 484/243, reaches zero gap in 4.842 s, and peaks at 390.90 MiB. At the connected 10K endpoint, the two selected-byte totals are 126,662,484/126,669,344 B; the 70K-variable, 115002-constraint solve takes 37.64 s, peaks at 287.14 MiB, retains a 3.654% certificate, and improves the objective by 7.339% over independent asset-then-plan selection.

These results expose the coverage transition. Coverage-first admission moves the required workload mass off the 80–100 ms fallback; within that set, CVaR selection is 0.45% faster than choosing the cheapest supports. On real-tag YFCC, the selected design uses base-derived bitmaps and online intersection, matching the measured sparse pair reuse.

Experiment 5: Mixed serving and joint design. We next ask whether one budgeted manifest improves mixed traffic and whether joint selection improves a frozen-admission two-stage policy. LAION1M supplies real caption tokens plus similarity, width, and height attributes. An outcome-independent query-by-example policy takes each query from one base row, which both ground truth and search remove. Each frozen process runs four cycles of 800 requests balanced across equality, conjunction, range, and same-attribute two-clause DNF. All arms retain identical state and scratch while enabling both route families (FULL), only direct scan (SCAN), only fragment composition (FRAG), or neither (LEAF).

Table 10: On frozen LAION traffic, FULL improves throughput and both tail latencies; joint selection also lowers latency relative to a frozen-admission two-stage control.

endpoint	strict/tie recall	QPS	p95 ms	p99 ms
FULL mixed (800 × 4)	1.000/1.000	1,680	10.58	17.36
LEAF mixed (800 × 4)	1.000/1.000	1,079	22.65	43.58
FULL / LEAF	—	1.557×	0.467×	0.398×
DNF U24 / LEAF (200)	1.000/1.000	1.596×	0.699×	0.627×
DNF Delta16 / LEAF (200)	1.000/1.000	1.591×	0.688×	0.637×

counterfactual	latency ratio	block range	<i>p</i>
frozen admission / joint	1.362×	1.295–1.484	1/64

Across eight fresh Latin-order blocks, all arms return bit-identical IDs and float32 distances at strict and tie-aware recall 1.000. FULL activates 800 direct and 1,600 fragment routes per block and, relative to LEAF, improves complete-wall QPS by 1.557× while reducing p95/p99 to 0.467/0.398×. The direct, fragment, and interaction contrasts are positive in every block. All eight blocks and 32 child processes complete under the prescribed structure, and FULL/LEAF throughput is positive in every block ($p = 1/256$). Identical objects, descriptors, scratch, and request order isolate dispatch and execution.

The complete runtime materializer seals four predicate families by four routing arms (16 cells) and audits all 800 exact supports before dispatch; every nonempty package fits the common optional-state cap. A fixed-route native gate then executes all 200 frozen DNF formulas in each of eight order-varied fresh-process blocks. Every arm receives one untimed warmup and one timed eight-lane batch per block. Direct U24 and Delta16 improve QPS over LEAF by 1.596× (95% CI [1.556, 1.637]) and 1.591× ([1.563, 1.620]); their p95/p99 latency ratios are 0.699/0.627 and 0.688/0.637. All 6,400 timed and 6,400 warm cells reproduce exact support IDs, top-10 IDs, and float32 distance bits. The intervention fixes the workload and changes only the selected support layout and native route.

The selected manifest contains 772 token-pair and 9,958 token-fixed-block supports. Its 13,475,748-byte state comprises 9,471,652 fragment bytes, a 4,000,000-byte norm vector, and a 4,096-byte

plan; a common 119,809-byte descriptor payload is outside optional-index accounting. Across 32 fresh processes, constructing the 10,730 supports and norm state takes 0.799–0.830 s before warmup, excluding base-file loading. FULL/LEAF terminal peak RSS is 2.461–2.495/2.615–2.734 GiB.

For the counterfactual, the frozen-admission two-stage control fixes fragments before compression, leaving compression savings unused by construction. Under the common 7,227,208-byte cap, the control and joint designs use 6,445,215/7,085,135 B; joint design co-selects 206 exact DNF supports, a sorted 16-bit-delta layout, and direct plans. Across six balanced crossover blocks and 24 processes, joint design reduces mean latency by 1.362× (1.295–1.484); all effects are positive ($p = 1/64$), with recall@10 1.000 and no invalid or duplicate IDs. Budgeted mixed serving therefore benefits both from activating the selected routes and from choosing fragments, layouts, and plans jointly.

6.4 Causal mechanisms

Experiment 6: Complete pair-support materialization. We first change only the availability of one complete pair support while holding atom state, router tuning, the 58-pair workload, $R=64$, the $b=640$ search path, and all non-pair state fixed. Each arm contributes five independent fresh OS processes, and log-Welch intervals use the two process populations. Separately, all 66 pair graphs are selected from base co-occurrence before evaluation; the coverage control uses all 58 non-benchmark identities, 100 distinct queries per pair, exact ground truth, and fixed $b=512$.

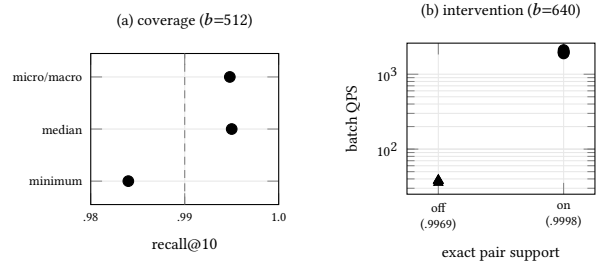


Figure 3: Selected pair supports maintain high recall across all 58 classes; enabling the exact support separates throughput by more than an order of magnitude.

The left panel of Figure 3 covers 5,800 held-out query vectors over supports spanning three orders of magnitude: minimum, median, and micro/macro recall are .984, .995, and .9948, and 54/58 classes reach .99. The pair-off arm retains the deployed approximate atom graph and uses exact intersection for its remainder, distinct from the all-exact fallback of Proposition 4. In the right panel, enabling the pair support improves pair-off/pair-on median latency by 27.16× [26.16, 28.19] and batch throughput by 52.74× [50.05, 55.58] while recall changes from .9969 to .9998. Pair-only is within 9.1% of pair-on in the retained single-process control. The fixed co-occurrence workload therefore isolates materialization rather than a router or atom-state change.

The same 5,800-query stream also compares complete systems at 100M scale. Across five fully warmed eight-thread processes

per system, Boole-ANN and Parlay-IVF achieve geometric-mean complete-batch throughputs of 2,048 and 124 QPS, a 16.54× advantage with 95% interval [15.63, 17.50], while strict recall rises from 0.99819 to 0.99981. The pair-only design stores 3.69 GB of pair-graph state; the complete Parlay-IVF index occupies 14.40 GB. Every returned ID passes predicate and duplicate checks.

Experiment 7: Support, layout, and plan controls. We next hold the SIFT100K queries, state, kernels, and 24-MiB cap fixed and disable one physical choice at a time. Table 11 reports five rotated blocks for support localization, contiguous layout, and range-prefix planning; its standalone conjunction frontier was analyzed in Section 6.2.

Table 11: Fixed-state controls isolate support localization, contiguous layout, and range-prefix planning.

control	blocks	FULL/control QPS	ratio range	geom. mean
support localization off	5	16,644 / 2,724	6.037–6.204	6.113×
contiguous layout off	5	16,644 / 3,731	4.441–4.504	4.469×
range prefix plan off	5	16,644 / 14,139	1.164–1.188	1.174×

Removing support localization, contiguous layout, or range-prefix pruning reduces mixed throughput by 6.113/4.469/1.174×; all 30 processes remain exact. The corresponding counters show 14.713× row-inspection amplification, 99.780% nonadjacent gathers, and 81.859% avoided suffix evaluations. Every arm retains 17,132,772 bytes of persistent state. FULL and plan-off charge a 4,096-byte transient upper bound, support-off and layout-off charge 408,192 bytes, all remain below the common cap, and mean peak RSS is 86,256–86,325 KiB. These controls attribute the SIEVE gap to localized fragments, their contiguous representation, and the selected prefix plan under an identical search kernel.

Experiment 8: Hierarchical range layout. Finally, we keep SIFT100K ranges, exact ground truth, thread count, and the searchable universe fixed and change only flat versus hierarchical block dispatch. The hierarchy is an overlapping dyadic extension whose 397 nodes use actual-byte accounting in place of the exactly- N membership accounting for a disjoint block family. Table 12 includes the β -WST reference and both Boole-ANN layouts.

Table 12: Hierarchical blocks reduce range fan-out from 101.20 to 8.46 calls per query and outperform β -WST.

System (8 threads)	recall@10	QPS	calls/query
β -WST	1.0000	2,954	—
Boole flat	0.9940	1,005	101.20
Boole hierarchy	1.0000	3,816	8.46

The flat width-500 layout reaches 0.994 recall and is 2.94× slower than β -WST at 101.2 leaf calls/query. A canonical cover over 397 dyadic supports, the window-tree organization shared with β -WST, reduces this to 8.46 calls. Across 64 resident q50 cycles, the hierarchy and β -WST both return all 32,000/32,000 exact-GT IDs; the hierarchy improves aggregate QPS by 1.292× and median cycle wall by 1.239×. Its complete 132.0 MB DAG, including overlapping

dyadic nodes, builds in 42.5 s and peaks at 665 MiB. Fixed shared ranges thus isolate the benefit of hierarchical dispatch.

7 CONCLUSION

Boole-ANN co-designs reusable supports, physical layouts, and recall-eligible plans; its exact support DAG and mandatory fallback preserve validity. It wins the selected YFCC high-recall and exact endpoints, every evaluated SIEVE family, and the qualified range/DNF comparisons. A controlled LAION intervention isolates joint selection from a frozen-admission two-stage policy.

ACKNOWLEDGMENTS

This work was supported by the National Key R&D Program of China (Grant No. 2024YFC3308200).

REFERENCES

- [1] Sanjay Agrawal, Surajit Chaudhuri, and Vivek R. Narasayya. 2000. Automated Selection of Materialized Views and Indexes for SQL Databases. In *Proceedings of the 26th International Conference on Very Large Data Bases*. 496–505.
- [2] Anas Ait Aomar, Karima Echihabi, Marco Arnaboldi, Ioannis Alagiannis, Damien Hilloulouin, and Manal Cherkaoui. 2025. RWalks: Random Walks as Attribute Diffusers for Filtered Vector Search. *Proc. ACM Manag. Data* 3, 3 (2025), Article 212. <https://doi.org/10.1145/3725349>
- [3] Abylay Amanbayev, Brian Tsan, Tri Dang, and Florin Rusu. 2026. Filtered Approximate Nearest Neighbor Search in Vector Databases: System Design and Performance Analysis. *arXiv preprint arXiv:2602.11443* (2026). <https://arxiv.org/abs/2602.11443>
- [4] Yuzheng Cai, Jiayang Shi, Yizhuo Chen, and Weiguo Zheng. 2024. Navigating Labels and Vectors: A Unified Approach to Filtered Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 6 (2024). <https://doi.org/10.1145/3698822>
- [5] Surajit Chaudhuri and Vivek R. Narasayya. 1997. An Efficient, Cost-Driven Index Selection Tool for Microsoft SQL Server. In *Proceedings of the 23rd International Conference on Very Large Data Bases*. 146–155.
- [6] Joshua Engels, Benjamin Landrum, Shangdi Yu, Laxman Dhulipala, and Julian Shun. 2024. Approximate Nearest Neighbor Search with Window Filters. *CoRR* abs/2402.00943 (2024). <https://arxiv.org/abs/2402.00943>
- [7] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *WWW*. DOI 10.1145/3543507.3583552.
- [8] Hao Guo, Jiwu Shu, and Youyou Lu. 2026. PipeANN-Filter: An Efficient Filtered Vector Search System on SSD. *arXiv preprint arXiv:2605.17992* (2026). <https://arxiv.org/abs/2605.17992>
- [9] Mengxu Jiang, Zhi Yang, Fangyuan Zhang, Guanhuo Hou, Jieming Shi, Wenchao Zhou, Feifei Li, and Sibao Wang. 2025. DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter. *Proc. ACM Manag. Data* 3, 3 (2025), Article 148. <https://doi.org/10.1145/3725399>
- [10] Zhaocheng Li, Silu Huang, Wei Ding, Yongjoo Park, and Jianjun Chen. 2025. SIEVE: Effective Filtered Vector Search with Collection of Indexes. *Proc. VLDB Endow.* 18, 11 (2025), 4723–4736. <https://doi.org/10.14778/3749646.3749725> <https://github.com/BillyZhaochengLi/SIEVE-vldb25>.
- [11] Zhonggen Li, Haoran Yu, Zixuan Xu, Yifan Zhu, and Yunjun Gao. 2026. A GPU-Accelerated Framework for Multi-Attribute Range Filtered Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2604.20121* (2026).
- [12] Yingfan Liu, Tong Wu, Jiadong Xie, Yang Zhao, Jeffrey Xu Yu, and Jiangtao Cui. 2026. Generalized Range Filtering Approximate Nearest Neighbor Search: Containment and Overlap. *arXiv preprint arXiv:2605.26474* (2026). <https://arxiv.org/abs/2605.26474>
- [13] Jiarui Luo, Miao Qiao, Chaoji Zuo, and Dong Deng. 2025. Tag-Filtered Approximate Nearest Neighbor Search. In *IEEE ICDE*. 3642–3654.
- [14] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proc. ACM Manag. Data* (2024). <https://doi.org/10.1145/3654923>
- [15] Zhencan Peng, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2025. Dynamic Range-Filtering Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 18, 10 (2025), 3256–3268. DOI 10.14778/3748191.3748193; <https://github.com/rutgers-db/DynamicSegmentGraph>.
- [16] Gaurav Sehgal and Semih Salihoglu. 2025. NaviX: A Native Vector Index Design for Graph DBMSs With Robust Predicate-Agnostic Search Performance. *Proc. VLDB Endow.* 18 (2025).

- [17] Harsha Vardhan Simhadri, Martin Aumuller, Amir Ingber, Matthijs Douze, George Williams, Magdalen Dobson Manohar, et al. 2025. Results of the Big ANN: NeurIPS'23 Competition. In *Advances in Neural Information Processing Systems 38 (Datasets and Benchmarks Track)*. <https://big-ann-benchmarks.com>.
- [18] Junjie Song, Yu Liu, Guoyu Hu, Zhongle Xie, Ming Yang, Beng Chin Ooi, and Ke Zhou. 2026. FAVOR: Efficient Filter-Agnostic Vector ANNS Based on Selectivity-Aware Exclusion Distances. *Proc. ACM Manag. Data* 4, 3 (2026), 1–25. <https://doi.org/10.1145/3802060>
- [19] Yitong Song, Bin Yao, Zhida Chen, Xin Yang, Jiong Xie, Feifei Li, and Mengshi Chen. 2025. Efficient Top-k Spatial-Range-Constrained Approximate Nearest Neighbor Search on Geo-Tagged High-Dimensional Vectors. *The VLDB Journal* (2025). <https://doi.org/10.1007/s00778-024-00894-5>
- [20] Ziqi Wang, Jingzhe Zhang, and Wei Hu. 2025. WoW: A Window-to-Window Incremental Index for Range-Filtering Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 6 (2025), 1–27. <https://doi.org/10.1145/3769843>
- [21] Tianming Wu and Dixin Tang. 2025. PathFinder: Efficiently Supporting Conjunctions and Disjunctions for Filtered Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2511.00995* (2025).
- [22] Jingyi Xi, Chenghao Mo, Benjamin Karsin, Artem Chirkin, Mingqin Li, and Minjia Zhang. 2025. VecFlow: A High-Performance Vector Data Management System for Filtered-Search on GPUs. *Proc. ACM Manag. Data* (2025). DOI 10.1145/3749189.
- [23] Haikexu, Guy Blelloch, Laxman Dhulipala, Lars Gottesbüren, Rajesh Jayaram, and Jakub Łącki. 2026. JAG: Joint Attribute Graphs for Filtered Nearest Neighbor Search. arXiv:2602.10258. Open-source: github.com/xuhaikexu/JAG.
- [24] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S. Jensen. 2024. iRangeGraph: Improvising Range-dedicated Graphs for Range-filtering Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 6 (2024). <https://doi.org/10.1145/3698814>
- [25] Chunxiao Ye, Xiao Yan, and Eric Lo. 2025. Compass: General Filtered Search across Vector and Structured Data. *arXiv preprint arXiv:2510.27141* (2025). <https://arxiv.org/abs/2510.27141>
- [26] Xiaowei Ye, Rong-Hua Li, Guoren Wang, Kaiwen Xue, Daiyin Wang, and Xubin Li. 2026. MCI: A Maximal Clique Index for Efficient Arbitrary-Filtered Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2604.22171* (2026). <https://arxiv.org/abs/2604.22171>
- [27] Yuanhang Yu, Dawei Cheng, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2026. Efficient Approximate Nearest Neighbor Search under Multi-Attribute Range Filter. *arXiv preprint arXiv:2602.15488* (2026). <https://arxiv.org/abs/2602.15488>
- [28] Zhiqiu Zou, Ziqi Yin, Rong-Hua Li, Hongchao Qin, Qiangqiang Dai, and Guoren Wang. 2026. RNSG: A Range-Aware Graph Index for Efficient Range-Filtered Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2603.12913* (2026). <https://arxiv.org/abs/2603.12913>
- [29] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 1 (2024), Article 69. <https://doi.org/10.1145/3639324>